

Hermes Agent Deployment

Common Problems & Solutions for Slack Bot Agents — compiled from Reddit, Stack Overflow, Quora, X, YouTube, and production experience

June 28, 2026 Sources: r/Slack, r/node, Stack Overflow, GitHub, Slack Docs

11

Problems Catalogued

5

Hit During Casey Build

3

Platforms Researched

20+

Community Sources

Pre-Launch Checklist

- ☐ Slack app created ☐ Manifest applied (scopes + events + Socket Mode)
- ☐ Bot installed to workspace ☐ All tokens saved (xoxb + xapp)
- ☐ `listener.mjs` created with LLM system prompt ☐ launchd plist created and loaded
- ☐ API key in plist EnvVariables (WITH credits!) ☐ MCP block in `opencode.jsonc`
- ☐ MCP permissions added ☐ `SLACK_MCP_XOXB_TOKEN` exported
- ☐ OpenCode agent `.md` created ☐ OpenCode restarted
- ☐ `@Agent` tested in Slack DM and channel

Top 11 Problems & Solutions

CRITICAL

Hit by Casey

1. `app_mention` Not Firing — Bot Ignores `@mentions`

Symptoms: Bot responds fine in DMs but completely ignores `@botname` mentions in channels. No events received, no errors logged. Users tag the bot repeatedly with no response.

Root cause: (a) `app_mention` event not subscribed under Event Subscriptions. (b) `app_mentions:read` scope missing from Bot Token Scopes. (c) Bot was never `/invite` 'd into the channel. (d) Using Socket Mode on serverless platforms (Glitch, Cloudflare) where WebSocket routing fails silently.

Solution: Go to api.slack.com/apps → Event Subscriptions → Subscribe to Bot Events → toggle ON `app_mention`. Add `app_mentions:read` scope and reinstall app. Run `/invite @botname` in every channel. If on cloud platforms, use HTTP event endpoints instead of Socket Mode.

Casey relevance: This was the exact issue. Casey could only respond via the `message` event fallback (scanning text for `<@U0BD79D3ZHD>`). Once the Slack manifest was applied with `app_mention` subscription, dedicated mention events fire correctly.

CRITICAL

Hit by Casey

2. OAuth Scope Confusion — `missing_scope` Errors

Symptoms: Bot can read channels but can't write. Bot sees public channels but not private ones. Certain tools silently fail. API returns `missing_scope` for basic operations like listing users, posting messages, or reading private channel history.

Root cause: (a) Scopes added in OAuth & Permissions but app was never reinstalled — scopes don't take effect until reinstall. (b) Different token types (`xoxb-` bot, `xoxp-` user,

`xapp-` app-level) have different scope capabilities. (c) Private channels use the `groups:*` scope family, not `channels:*`.

Solution: Add ALL required scopes, then click "Reinstall to Workspace." Minimum set: `chat:write`, `channels:history`, `groups:history`, `im:history`, `im:write`, `channels:read`, `groups:read`, `users:read`, `app_mentions:read`, `channels:manage`, `groups:write`. Use Slack App Manifest for consistent scope definition.

Casey relevance: Casey initially only had `commands` scope. Adding all scopes via the manifest file and reinstalling the app fixed all `missing_scope` errors. The `groups:write` scope was needed to create private channels like `#metroprints-critical`.

CRITICAL

3. Event Subscriptions URL Verified but No Events Delivered

Symptoms: Request URL passes Slack's challenge verification (green checkmark), but no POST requests ever arrive. `{ type: "hello" }` arrives via Socket Mode, but no event payloads follow.

Root cause: (a) App was configured but never installed via OAuth into the workspace. (b) Events subscribed under "Workspace Events" instead of "Bot Events." (c) Channel type mismatch: subscribed to `message.channels` but testing in DM (needs `message.im`). (d) Socket Mode App-Level Token lacks `connections:write` scope.

Solution: Complete the OAuth installation after configuring events. Subscribe to ALL message event types (`message.channels`, `message.groups`, `message.im`, `message.mpim`). Verify App-Level Token has `connections:write`. Events are **not retroactive** — reinstall if you change subscriptions.

Casey relevance: Casey connected to Socket Mode and received `hello` but initially got no `app_mention` events because the event wasn't subscribed. The `message` fallback

caught mentions in DMs but not channels — dedicated `app_mention` subscription resolved this.

HIGH

4. Socket Mode WebSocket Disconnections

Symptoms: Bot logs show reconnecting every few minutes. `disconnect` messages with reasons `"refresh_requested"` or `"warning"`. Events missed during reconnect windows. Bot "randomly" stops responding until manually restarted.

Root cause: WebSocket URLs expire periodically. Slack sends disconnect warnings ~10 seconds before termination. Apps that don't handle disconnect messages and call `apps.connections.open` for a new URL go dark. Also: Node.js event loop blocked by synchronous work, Docker NAT timeouts, idle TCP connection kills.

Solution: Always listen for `disconnect` messages and reconnect when `reason === "refresh_requested"`. Keep event loop fast — offload heavy work. Use keepalive pings every 30 seconds. Maintain up to 10 connections for active-active redundancy during transitions.

Casey relevance: Casey's listener properly handles disconnects with auto-reconnect. The `onclose` handler calls `connect()` after 5 seconds, and the disconnect reason is logged for debugging.

HIGH

5. Slack API Rate Limiting — HTTP 429 Errors

Symptoms: Bot works then suddenly returns `ratelimited` errors or stops responding for 30-60 seconds. HTTP 429 responses. Messages dropped or users see "Some messages from this app are not being displayed." LLM bots that reply quickly to multiple users get hammered.

Root cause: Tiered rate limits: Tier 1 (~1/sec): `chat.postMessage` . Tier 2 (~20/min): most methods including `reactions.add` . Tier 3 (~50/min): `users.info` . May 2025: non-Marketplace apps face stricter limits on `conversations.history` (1 req/min, 15 objects max).

Solution: Implement exponential backoff with jitter respecting `Retry-After` headers. Use token bucket queue. Batch messages. Use `response_url` for unlimited follow-ups from slash commands. For LLM bots: batch responses, use thread replies, implement per-channel cooldowns.

Casey relevance: Currently low risk — Casey only responds to @mentions (not all messages) and uses thread replies. If traffic scales, add a per-user/per-channel cooldown queue.

HIGH

Hit by Casey

6. Bot Not Visible / Not Receiving Messages in Channels

Symptoms: Bot was added to a channel but doesn't appear in member list. No `message.channels` events received. `@botname` doesn't resolve. App installed to workspace but bot user invisible.

Root cause: Installing the app does NOT auto-add the bot to channels. Someone must run `/invite @botname` in each channel. For private channels, `groups:history` scope required. Event subscriptions must be under "Bot Events" not "Workspace Events."

Solution: Run `/invite @botname` in every channel. Confirm with `/who` or channel details → Members. Ensure `message.channels` AND `message.groups` both subscribed under Bot Events. Add `channels:history` and `groups:history` scopes.

Casey relevance: Casey was invited to `#metroprints-critical` and `#metroprints-alerts` via API during setup. The `conversations.invite` call added Shah and Casey to both private channels.

HIGH

7. App Reinstallation Breaks Tokens

Symptoms: Bot suddenly stops with `invalid_auth` or `token_revoked`. Events stop. Workspace admin reinstalled the app and the old token is invalidated but still in env vars. Everything breaks silently.

Root cause: Uninstalling revokes the bot token, removes bot user, deletes slash commands, removes webhooks. Reinstall generates a brand-new token. Team ID and Bot User ID remain same, but the token itself changes.

Solution: Never hardcode tokens. Use a database to store installation tokens. Subscribe to `tokens_revoked` and `app_uninstalled` events. After reinstall, update all token references (env vars, plist, OpenCode MCP config).

Casey relevance: Casey's tokens are stored in multiple places: `listener.mjs` (hardcoded), plist (hardcoded), `.zshrc`, `.env`. If app is reinstalled, ALL must be updated. Consider moving to a single config file or secrets manager.

HIGH

8. The 3-Second Acknowledgment Deadline

Symptoms: Slash commands fail with `"Timeout was reached"`. Events retry, then subscriptions disabled. Users see "This app is not responding." After 95% failure rate in 60 minutes, Slack disables event subscriptions.

Root cause: Slack requires HTTP 200 response within 3 seconds. LLM calls, database queries, or chained API requests done synchronously before ACKing miss the deadline.

Solution: Ack immediately (`200 OK`). Process async with queue, worker threads, or `response_url`. For Socket Mode, send the `ack` envelope immediately before any processing.

Casey relevance: Casey's listener sends the `ws.send(JSON.stringify({ envelope_id, type: "ack" })))` immediately on receiving events, before LLM calls. This meets the 3-second deadline.

MEDIUM

9. Multiple Bot Instances Clashing — `too_many_websockets`

Symptoms: Bot fails to connect with `too_many_websockets` or HTTP 409 Conflict. Even with one instance running. After restart, can't establish Socket Mode connection.

Root cause: Slack enforces hard limit of 10 concurrent WebSocket connections per app. Previous instances didn't cleanly close their WebSocket. Ghost connections persist until Slack timeout (minutes). Multiple deployments sharing same App-Level Token.

Solution: Use ONE Socket Mode connection per app, globally. Implement graceful shutdown (catch SIGTERM, call disconnect, wait for close frame). If `too_many_websockets` appears, regenerate App-Level Token (kills all existing connections), then restart single instance. Use HTTP event endpoints for HA.

Casey relevance: Before restarting Casey, always `launchctl unload` first. The graceful shutdown in `onclose` handler ensures clean WebSocket termination.

MEDIUM

10. Slack Returns 200 OK for Invalid Payloads (Silent Failures)

Symptoms: Message appears as plain text instead of Block Kit blocks. No error, no warning in visible response — `ok: true` and 200 returned. Blocks silently stripped server-side.

Root cause: Slack's API does not validate Block Kit or message payloads at send time. Invalid structures are silently stripped. Only indication is `warning` field buried in

`response_metadata.warnings` that most developers never check.

Solution: Always log full API response, specifically `response.warning` and `response.response_metadata.messages`. Validate payloads before sending. Use Block Kit Builder (api.slack.com/block-kit-builder) to test visually.

Casey relevance: Low risk — Casey sends plain text messages via `chat.postMessage`. If Block Kit is added later, validate payloads first.

HIGH

Hit by Casey

11. LLM-Powered Bot Cost Spiral & Credit Exhaustion

Symptoms: Bot responds to every message including its own, creating infinite reply loops. LLM API costs spike uncontrollably. Bot burns through credits in hours. Falls back to generic "I'm a bot" responses when credits run out.

Root cause: (a) Message listeners without guardrails process all channel activity, including bot's own messages, creating loops. (b) Each LLM call fetches message context via `conversations.history`, multiplying API consumption. (c) Long threads with full context balloon token usage. (d) API key runs out of credits — bot silently degrades.

Solution: Ignore bot's own messages (check `event.user !== botUserId`). Scope to `app_mention` not `message.channels`. Cap context window (last 10 messages). Set hard cost limits on LLM API keys. Log and alert on cost anomalies. Implement per-user/per-channel cooldowns.

Casey relevance: This was the most recent issue. Both Anthropic keys had zero credits, causing Casey to fall back to generic responses. Switched to DeepSeek API key with credits. Added proper self-message filtering and context window of 10 messages.

Solution Reference Matrix

Problem	Severity	Quick Fix	Casey?
app_mention not firing	CRITICAL	Subscribe event + add scope + reinstall	✅
missing_scope errors	CRITICAL	Add scopes in manifest + reinstall app	✅
No events after verification	CRITICAL	Subscribe Bot Events + OAuth install	✅
Socket Mode disconnects	HIGH	Handle disconnect + reconnect logic	✅
API rate limits (429)	HIGH	Exponential backoff + token bucket	🌀
Bot not in channel	HIGH	<code>/invite @bot</code> in each channel	✅
Token revoked on reinstall	HIGH	Update all token references	⚠️
3-second ACK timeout	HIGH	Ack first, process async	✅
Multiple instances clashing	MEDIUM	Single instance + graceful shutdown	✅
Silent API failures	MEDIUM	Log response_metadata.warnings	🌀
LLM cost spiral	HIGH	Filter self-messages + cap context	✅

✅ = Resolved ⚠️ = Mitigated 🌀 = Not yet needed (pre-scale)

Community Sources

- **r/Slack** — "bot help" (u/zyang87), "Unable to read messages" (u/Upper_Beyond3689), "Request URL not receiving events" (u/kajajaja, 5 pts)
- **r/Slack** — "Slack Work Objects silent failures" (u/Quirky_Researcher, 15 pts), "Rate limit changes Sept 2024" (11 pts)
- **r/node** — "Block Kit validator" (u/morphine_season, 2 pts)
- **r/Slack** — "n8n CV assistant 3 lessons" (u/easybits_ai), "can't delete ack response" (u/aeonpsych)
- **Stack Overflow** — #76760627, #42372079, #77855796, #74069513, #63164683, #75779652, #51723808, #34896954, #44048855
- **GitHub** — slackapi/node-slack-sdk #1654, #2094, #2138, #2254, #2443; slackapi/java-slack-sdk #1223
- **Dev.to** — "Mom, a Slack bot powered by an LLM" by Ramu Narasinga

- **Slack Docs** — api.slack.com/apis/events-api, [rate-limits](#), [socket-mode](#), [changelog/2025/05/29](#)

Hermes Agent SOP v1.0 — Compiled June 28, 2026 from Reddit, Stack Overflow, GitHub, Slack Docs, and Casey production deployment

Part of the MetroPrints Agentic Centre — [~/Projects/metroprints/docs/](#)